第35课 DFS入门

一、课程信息

项目	内容
课次	第 35 课 / 共 40 课
阶段	阶段四：算法思想（第 1 课）
主题	DFS（深度优先搜索）入门
前置知识	递归、函数、数组
课时	100 分钟

教学目标（可检验）

1. **能说出** DFS 的全称和核心思想，能用自己的话解释"一条路走到黑，走不通再退回来"。
2. **能手画** 3 个数全排列的 DFS 搜索树，标注每一步的选择与回溯。
3. **能独立编写** 输出 1\~N 全排列的 DFS 程序，正确使用访问标记数组和回溯（恢复状态）。

二、教学重点与难点

重点

- DFS 的核心思路："递归 + 回溯"。
- 访问标记数组 `used[]` 的作用：记住"哪些东西已经用过了"。
- 回溯操作：递归返回后要**恢复现场**（把标记清除、把选的东西撤掉）。

难点

- **回溯的理解**：为什么递归回来后要"擦掉"之前的标记？孩子容易忘记恢复状态。
- **搜索树的建立**：从代码到搜索树、从搜索树到代码，两个方向都要能对应。
- **递归层数与排列位置的对应关系**：第几层递归 = 正在填第几个位置。

三、课堂时间分配（共 100 分钟）

时间	时长	环节	内容
0′–5′	5 min	开场导入	生活故事：走迷宫的策略
5′–20′	15 min	新知讲解①	DFS 是什么？手动模拟"3 个数全排列"
20′–30′	10 min	画图互动	一起画搜索树，标注选择与回溯
30′–50′	20 min	例题①	全排列代码逐行讲解 + 运行演示
50′–55′	5 min	休息	休息、喝水
55′–70′	15 min	例题②	从 N 个数中选 M 个排列（稍加变化）
70′–90′	20 min	课堂练习	学生独立完成 2–3 道练习题
90′–98′	8 min	总结答疑	常见错误 + 核心口诀 + 答疑
98′–100′	2 min	布置作业	课后题 + 下节课预告

> **讲解总时长**：15 + 20 + 15 = 50 分钟（单次连续讲解不超过 20 分钟）
> **练习总时长**：10 + 20 = 30 分钟

四、讲稿提纲

4.1 开场导入（5 分钟）——走迷宫的故事

> 同学们，假设你走进了一个大迷宫，面前有三条路。你会怎么办？
> 有人说"随便挑一条走"——没错！但如果走到死胡同了呢？
> 你会**退回来**，换另一条路继续走。这就是今天要学的 **DFS——深度优先搜索**！
> 它的策略就一句话：**"一条路走到黑，走不通再退回来换一条。"**

板书关键词：DFS = Depth First Search = 深度优先搜索

4.2 新知讲解①（15 分钟）——用生活例子理解 DFS

故事 1：搭配衣服

> 小明有 3 件上衣，编号 1、2、3。妈妈说："你把这 3 件衣服排成一排，看看有多少种排法？"
>
> 小明这样做：
> - 先拿 1 号放第一个位置
> - 再拿 2 号放第二个位置
> - 剩下 3 号放第三个位置 得到排列 **1 2 3**
> - 退回来，第二个位置换成 3 号
> - 剩下 2 号放第三个位置 得到排列 **1 3 2**
> - 退回来，第一个位置换成 2 号……
>
> 这个过程，就是 DFS！

关键概念逐一引出：

1. **递归**：每一层递归 = 正在填第几个位置（第 1 个位置 第 2 个位置 第 3 个位置）。
2. **标记数组 `used[]`**：就像小明把拿走的衣服做了个记号，告诉自己"这件用过了，不能再拿"。
3. **回溯**：就像小明把衣服放回衣柜、撕掉记号，表示"这件我不选了，要换别的试试"。

故事 2：点菜

> 如果你去餐厅，菜单上有 A、B、C 三道菜，服务员说"请按你最想吃的顺序排一下"，你怎么排出所有可能的顺序？——方法一模一样！

4.3 画图互动（10 分钟）——搜索树

在黑板上画出 **3 个数全排列的搜索树**：

```
                开始（还没选）
                 /        |        \
            选1        选2        选3          第1个位置
              / \        / \        / \
        选2   选3   选1   选3   选1   选2       第2个位置
          |     |     |     |     |     |
        选3   选2   选3   选1   选2   选1       第3个位置

          123   132   213   231   312   321        6种排列
```

互动环节：

- 请一位同学上台，在搜索树上用红笔描出"从 123 回溯到 132"的路径。
- 问同学们："如果是 4 个数的全排列，搜索树第一层有几个分支？"（答：4 个）
- "4 个数一共有多少种排列？"（答：4×3×2×1 = 24 种）

4.4 代码讲解过渡

> 好，我们已经用手把 DFS 的过程模拟出来了。现在我们来教计算机做同样的事情！
>
> 计算机比我们快得多——3 个数排列有 6 种，4 个数有 24 种，10 个数呢？有 3628800 种！人手排不完，但计算机一秒就能搞定。

七、常见错误与应对

错误 1：忘记回溯（最最常见！）

```
//  错误写法：缺少 used[i] = false;
if (!used[i]) {
    arr[step] = i;
    used[i] = true;
    dfs(step + 1);
    // 忘记写 used[i] = false; !!!
}
```

现象：只输出一行结果（如 `1 2 3`），然后程序就结束了。

诊断方法：告诉学生——"如果你的程序只输出了一种排列，八成是忘了回溯！"

应对口诀：**"标记和取消是一对好朋友，写了 `true` 一定记得写 `false`！"**

错误 2：数组下标从 0 开始导致混乱

```
//  容易混乱的写法
void dfs(int step) {
    if (step >= n) { ... }  // 到底是 >= n 还是 > n？
    ...
}
```

应对：统一约定——**数组下标从 1 开始**，`dfs(1)` 表示填第 1 个位置，`step > n` 表示填完了。保持一致就不容易错。

错误 3：递归出口写错位置

```
//  错误：把输出写在了 for 循环里面
for (int i = 1; i <= n; i++) {
    if (!used[i]) {
        arr[step] = i;
        used[i] = true;
        if (step == n) {  // 在这里输出——虽然能跑，但不规范
            // 输出...
        }
        dfs(step + 1);
        used[i] = false;
    }
}
```

应对：递归出口**永远写在函数最前面**，形成习惯。先判断"该不该停下来"，再决定"继续往下走"。

错误 4：搞混"排列"和"组合"

现象：学生问"为什么 `1 2` 和 `2 1` 算两种？"

应对："排列"关心**顺序**——站队的时候，小明站前面和小红站前面是不一样的！（组合的概念留到以后再讲。）

错误 5：`used` 数组没有初始化

现象：全局数组默认为 `false/0`，局部数组可能有垃圾值。

应对：**建议统一使用全局数组**（全局变量自动初始化为 0），避免踩坑。这个在竞赛中也是常用写法。

九、下节课预告

> 同学们，今天我们学会了 DFS 的基本套路——递归 + 回溯。用它来生成全排列，是不是很酷？
>
> 下节课（第 36 课），我们要把 DFS 用到**更好玩的地方**：
>
> 1. **走迷宫**：给你一个地图，用 DFS 找到从起点到终点的路径！
> 2. **八皇后问题（简化版）**：在棋盘上放皇后，让她们互不攻击——这是一道超级经典的搜索题！
>
> 今天学的全排列模板就是基础，下节课只是在上面"加条件"。所以今天的代码一定要**自己多敲几遍**，敲到闭着眼睛都能写出来！
>
> 我们下节课见！

附录：DFS 模板速记卡

```
┌──────────────────────────────────────────────┐
│            DFS 全排列模板口诀                 │
│                                              │
│  1. 递归出口先写好，填满就输出                 │
│  2. for 循环逐个试，没用过的才能选             │
│  3. 选了就标记 true，递归填下一个              │
│  4. 回来记得 false，回溯不能忘!               │
│                                              │
│  口诀：选、标、递、消 —— 四步走天下            │
│                                              │
│  选：arr[step] = i;                          │
│  标：used[i] = true;                         │
│  递：dfs(step + 1);                          │
│  消：used[i] = false;                        │
└──────────────────────────────────────────────┘
```

教案编写日期：2026 年 4 月 10 日